

Tightly Integrated Addons in Alibre Design (in C++)

Tight integration with Alibre Design requires that an add-on implement a COM interface named `IALibreAddOn`. The pointer to this interface will be obtained by Alibre Design at the time when Alibre Design loads the add-ons when it is launched. Alibre Design does this by invoking the function `GetAddOnInterface ( )` exported by the add-on's DLL. All advanced add-ons desiring tight UI integration with Alibre Design will have to implement this function and return the `IALibreAddOn` interface pointer implemented by the add-on.

1. Exposing Addon UI in Alibre Design

1.1 Defining add-on's menu commands

Add-ons can expose a menu structure within Alibre Design by implementing certain methods of the interface, `IALibreAddOn`. When user clicks on an add-on command, `AlibreDesign` will call into the add-on in order to activate that command. An activated command can remain active until terminated. A command can be implemented in a few different ways. Details on this are explained in Section 2 below.

1.2 Adding dedicated ribbon tab in Alibre Design's Ribbon control

It is also possible for an add-on to display its menu commands under a dedicated ribbon tab in Alibre Design's ribbon (see method `UseDedicatedRibbonTab` in `IALibreAdOn` interface)

1.3 Adding GUI elements within Alibre Design's Design Explorer

An add-on command can display its UI elements (say, a dialog panel) within Alibre Design's Design Explorer.

The requirements from an add-on in order to implement such integrated UI are outlined below:

- An add-on command can insert its own custom tab-page (panel) in Alibre Design's Design Explorer. It can then embed its UI elements there. The add-on command has to implement `IAlibreAddOnCommand` interface.
- When the add-on command is invoked by user, Alibre Design will call `IAlibreAddOnCommand.AddTab (VARIANT_BOOL *pAddTab)` on the command. If the add-on does not require a GUI presence, it will return `E_NOTIMPL` or set `VARIANT_FALSE` to `*pAddTab` and a tab page will not be added. Else a new tab page will be created for the add-on by Alibre Design.
- If the add-on command indicated to Alibre Design that it wanted to add a tab page to the Explorer pane, Alibre Design will subsequently call the method `IAlibreAddOnCommand.OnShowUI(int hWnd)` on the command. Here, the add-on needs to add whatever UI controls it wants to show inside this tab page. The `hWnd` passed in is the windows handle of the newly added pane for Add-on for parenting purposes

1.4 Adding a dedicated pannel docked to Alibre Design's graphics window

Additionally, an add-on command can open its own dedicated panel that is shown docked with Alibre Design's Graphics window. The add-on command can show any custom user interface within such a panel (examples include for instance, a dialog panel, or a text editor or custom graphics or a combination of these). For more information refer to `IADAddOnCommandSite` interface or a sample add-on that demonstrates this.

1.5 Rendering graphics into Alibre Design's graphics window

An add-on can draw into AD's Geometry canvas (graphics window). Let us lay here the requirements and assumptions that define this feature:

- An add-on requiring this capability will have to support the `IAlibreAddOn` interface.
- The capability involves drawing or superimposing graphic elements (lines, text and other symbols) into AD's graphics window that is tightly synchronized with AD's rendering process.
- This capability will be available to an add-on only within the scope of executing an add-on menu command that implements `IAlibreAddOnCommand`. Alibre Design will call the 'OnRender/OnRender3D' methods on this interface during the lifetime of the command whenever Alibre Design is updating its graphics window.

- The graphics rendered by the add-on are of transient nature. That is, they can be displayed only as long as the add-on command is active. So, this capability is ideal for temporary graphics like previews and operation results etc. – things that do not stick around after the lifetime of the command that drew them.
- The active add-on command can trigger update of Alibre Design's display canvas. This can be achieved by explicitly calling method UpdateCanvas or InvalidateCanvas on the IADAddOnCommandSite interface telling AD to redraw its canvas.
- It is possible for the add-on to instruct Alibre Design to not perform its (the latter's) standard rendering during an update triggered by the add-on. During a canvas update, this will cause Alibre Design to clear the canvas and then let the add-on alone to render its graphics to the canvas via the call to 'OnRender/OnRender3D' mentioned above.

1.6 Saving custom data to Alibre file

Add-on can persist data within Alibre Design file which is a compound file. AddOn interested in persisting its data within Alibre Design file needs to provide additional XML tag in the configuration file (<AddOn>.adc) file. Following is the sample XML tag:

```
<PersistentData streamName="MyStreamName"/>
```

Additionally, the add-on will have to implement two methods on IAlibreAddOn interface, namely, LoadData () and SaveData ().

2 Steps to implement tightly integrated AddOn in Visual C++

Let us take a closer look at implementing the interfaces IAlibreAddOn and IAlibreAddOnCommand that AlibreDesign would expect an AddOn wanting tighter integration to support.

Step 1: Create a new VC++ DLL project, include the AlibreAddOnX.h in the StdAfx.h with the path suitable to your machine:

```
// Edit the folder path to the type library files below based on where Alibre Design is
// installed on your computer
#import "C:\Alibre Design\Program\AlibreX_64.tlb"
#import "C:\Alibre Design\Program\AlibreAddOn_64.tlb" raw_interfaces_only

using namespace AlibreX;
using namespace AlibreAddOn;
```

The typelibrary “AlibreAddOn_64” is required for integrating your add-on inside Alibre Design. This is the main topic of this document.

Typelibrary “AlibreX_64” exposes API to drive functionality in Alibre Design. For instance, API to query/manipulate geometry or assembly structure in an Alibre Design model. Note that this API set is different and more extensive than the add-on API set noted above.

Step 2: Implement a function: GetAddOnInterface () and export it on the AddOn’s DLL.

An Alibre add-on DLL has to export these functions: AddOnLoad, AddOnInvoke, and AddOnUnload. These are mandatory for any AddOn in Alibre. In addition to these functions, the function GetAddOnInterface needs to be implemented and exported by the DLL of an add-on requiring tighter integration with Alibre Design. This DLL function will have to return object of type: IAlibreAddOn Interface as IUnknown*.

Following is the typical definition of these exported functions in the main header file in the AddOn DLL project:

```
extern "C"
{
    APICLIENTAPP_API void AddOnLoad (HWND windowHandle,
                                     VOID *pAutomationHook,
                                     VOID *reserved);

    APICLIENTAPP_API void AddOnInvoke (HWND windowHandle,
                                       VOID *pAutomationHook,
                                       LPCSTR sessionName,
                                       BOOL isLicensed,
                                       VOID *reserved1,
                                       VOID *reserved2);

    APICLIENTAPP_API void AddOnUnload (HWND windowHandle,
                                       BOOL forceUnload,
                                       BOOL *cancel,
                                       VOID *reserved1,
                                       VOID *reserved2);

    APICLIENTAPP_API IUnknown* GetAddOnInterface ();
}
```

The implementation of GetAddOnInterface () function should instantiate an object of class, say, CAddOnInterface, that is derived from IAlibreAddOn (more on this in Step 3 below). It will return this to the caller (which of course will be Alibre Design) as an IUnknown pointer. Typical Implementation of GetAddOnInterface () function could look like this:

```

APICLIENTAPP_API IUnknown* GetAddOnInterface ()
{
    AFX_MANAGE_STATE(AfxGetStaticModuleState())

    IUnknownPtr result = NULL;
    CSampleAddOnInterface* addOnInterface = new CSampleAddOnInterface ();

    // store a reference to the COM interface in the application object (will result
    // in incrementing reference count on the interface)
    if (theApp.m_pAddOnInterface == NULL)
        theApp.m_pAddOnInterface = addOnInterface; //

    // another reference count will be added as we return the interface to caller
    result = (IUnknown *)addOnInterface;

    return result;
}

```

Note: Root object, “theApp.m_pRoot”, in above function is the IADRoot object that can be obtained from AutomationHook in AddOnLoad () function. IADRoot is the top-most API object in Alibre Design’s Automation API hierarchy. See the sample addon project for details.

Step 3: As mentioned above, we add a new class, say, CAddOnInterface to implement IAlibreAddOn. The definition of this class in its header file looks like this:

```

class CSampleAddOnInterface : public IAlibreAddOn
{
public:
    CSampleAddOnInterface();
    virtual ~CSampleAddOnInterface();

public:
    // Methods from IAlibreAddOn
    HRESULT _stdcall get_RootMenuItem (/*[out,retval]*/ long *pRootMenuID);

    HRESULT _stdcall HasSubMenus (/*[in]*/ long menuID,
    /*[out,retval]*/ VARIANT_BOOL *pHasSubMenus);

    HRESULT _stdcall SubMenuItems (/*[in]*/ long menuID,
    /*[out,retval]*/ SAFEARRAY **pSubMenuIDs);

    HRESULT _stdcall MenuItemText (/*[in]*/ long menuID,
    /*[out,retval]*/ BSTR* pMenuDisplayText);

    HRESULT _stdcall HasPersistentDataToSave(/* [in] */ BSTR sessionIdentifier,
    /*[retval][out] */ VARIANT_BOOL *IsPopup);

    HRESULT _stdcall PopupMenu (/*[in]*/ long menuID,
    /*[out,retval]*/ VARIANT_BOOL *IsPopup);
}

```

```

HRESULT _stdcall MenuItemState (/*[in]*/ long menuID,
/*[in] */ BSTR sessionIdentifier,
/*[out, retval]*/ enum ADDONMenuStates *pType);

HRESULT _stdcall MenuItemToolTip (/*[in]*/ long menuID,
/*[out, retval]*/ BSTR *pToolTip);

HRESULT _stdcall InvokeCommand (/*[in]*/ long menuID,
/* [in] */ BSTR sessionIdentifier,
/*[out, retval]*/ IAlibreAddOnCommand **pCommand);

HRESULT _stdcall LoadData (/*[in]*/ struct IStream * ppCustomData,
/*[in]*/ BSTR sessionIdentifier);

HRESULT _stdcall SaveData (/*[in]*/ struct IStream * pCustomData,
/*[in]*/ BSTR sessionIdentifier);

HRESULT _stdcall setIsAddOnLicensed (/*[in]*/ VARIANT_BOOL isLicensed);

HRESULT _stdcall MenuIcon(long, BSTR *); /* api available only if
'specificationVersion' in .adc file is greater than "1" */

HRESULT _stdcall UseDedicatedRibbonTab(VARIANT_BOOL *); /* api available only if
'specificationVersion' in .adc file is greater than "1" */

```

Important Note: The methods **MenuIcon** and **UseDedicatedRibbonTab** above are applicable only when the **'specificationVersion'** attribute's value is greater than "1" in the add-on's configuration file (.ADC) as these were introduced in version 2 of AD add-on interface specification. **MenuIcon** is called by Alibre Design to query the name of an add-on supplied icon file that is associated with a specific add-on command. **UseDedicatedRibbonTab** is called by Alibre Design to find out if the add-on wants to display its command buttons under a dedicated tab in Alibre's ribbon control. The add-on framework's specification version is "2" at the time of last update of this document.

See the sample add-on project regarding implementing the standard IUnknown and IDispatch interfaces.

Let us discuss about the necessary implementation for required **IAlibreAddOn** interface methods:

IAddOnInterface.get_RootMenuItem (int *pRootMenuID) will need to return a non-zero integer value in **pRootMenuID**. If Root Menu ID returned is zero, or if the method **IAddOnInterface.get_MenuItemText (int RootMenuID, BSTR* pMenuDisplayText)** returns an empty string, Alibre Design will not attempt to call any of the other methods needed to build a menu structure for the add-on to show on Alibre Design's menu/ribbon.

If Alibre Design receives a non-zero value for Root Menu ID, it queries for its sub menu items. For each menu id, Alibre Design recursively calls **get_HasSubMenus (menuID, pHasSubMenu)**, **IAddOnInterface::get_SubMenuItems (menuID, ppSubMenuIDs)** and **IAddOnInterface::getMenuItemText (menuID, pMenuDisplayText)** to construct the menu hierarchy for the add-on.

When user clicks the add-on's root menu to expand it, Alibre Design updates the state of the Menu item by calling **IAlibreAddOn.getMenuItemState (menuId)**. Depending on the Menu State (one of enum constants defined in **ADDONMenuStates**) returned by the method, menu is shown enabled or disabled etc.

When user clicks any of the add-on's leaf menu commands, Alibre Design calls **IAlibreAddOn.InvokeCommand (/*[in]*/ int menuId, /* [in] */ BSTR sessionId, /*[out, retval]*/ IAlibreAddOnCommand **pCommand)** on the add-on. Implementation of this function in your class, say, **CAddOnInterface**, would handle the command processing. For implementing a command that needs to process user events like mouse clicks, or needs to show GUI controls etc, **InvokeCommand ()** will have to return an object of type: **IAddOnCommand**. If the returned object is not NULL and is of type **IAddOnCommand**, AlibreDesign will create a **command site object** for the command and set the **CommandSite** property on this interface. The command site object represents this add-on command on Alibre Design's side and provides a means for the add-on command to define command behavior, terminate command etc. Alibre does this by implementing the interface, **IADAddOnCommandSite**, on command site object. Note that the lifetime of the command site object is the life time of the command itself. That is, it goes away when the command is terminated. We will now talk about the various flavors of commands an add-on can implement.

An add-on can implement a command to be:

- 1) A two-way toggle command having just two states – active and inactive. As an example, such a command could be made to display an add-on specific tab window in Alibre Design's Explorer pane. Usually, the very first command that user selects to activate or launch an add-on is implemented as a two-way toggle command.
- 2) A regular command having multiple states that can listen to Alibre Design events, display graphics on Alibre Design's graphics windows and which will get terminated upon user invoking any other command. More on this in the next section, Step 4.
- 3) A stateless, 'fire and forget' type command that just has to perform simplistic processing without any advanced integration requirements. For such a command, the add-on would implement **IAlibreAddOn.InvokeCommand ()** method to do all the needed processing so that when this method returns, the command has been fully executed. That is, lifetime of the command is the duration of the execution of **InvokeCommand ()**. **InvokeCommand** would return NULL back to Alibre.

Note: If `IAlibreAddOn.get_RootMenuItem()` returned zero, AddOn will be listed under Tools → AddOns menu as also the ribbon under the “Add-on” ribbon tab. When user clicking on this menu item, Alibre Design will call `IAlibreAddOn.InvokeCommand()`. Subsequent behavior of the command will depend on the implementation of AddOn’s `IAlibreAddOn.InvokeCommand()` as described above. If this command returns an `IAlibreAddOnCommand` object, then, the usual process of Showing UI takes place as mentioned earlier.

Now, let us discuss how an add-on can save its add-on specific data into Alibre Design’s workspace. First of all, such add-ons need to expose a special XML tag entry, “Persistent Data” in the add-on configuration (.adc) .The “StreamName” property value should be a valid string. Sample entry:

```
<PersistentData StreamName="SampleAddOn_Data"/>
```

During the save process, for AddOns with above-mentioned entry in its .adc file, Alibre Design will call, `IAlibreAddOn.getHasPersistentDataToSave (BSTR sessionId, VARIANT_BOOL *pboolHasData)`. If true is returned, an IStream with this name will be created in the Alibre file (which is a compound file). Alibre Design will then call `IAlibreAddOn.SaveData (struct IStream * pCustomData, BSTR sessionId)` passing in this IStream pointer. Add-on should implement code to write its data to this IStream. Alibre Design takes care of committing the IStream to disk.

Conversely, upon opening any Alibre Design model file, if any loaded AddOn has the above-mentioned entry in its .adc file, then, Alibre Design will attempt to access the IStream of given name. If the IStream is found, then, Alibre Design calls into the add-on to give the add-on to load the contents of its private data contained in its stream:

`IAlibreAddOn.LoadData (struct IStream * pCustomData, BSTR sessionId)`

IMPORTANT NOTE:

In several of the methods on the interface `IAlibreAddOn`, notice that there is an argument named “**sessionId**” that Alibre Design passes to the add-on. Alibre Design uses this string as means of conveying to the add-on, the session (workspace) that this call is associated with. This is because, in the Alibre Design process, user could have opened several workspaces (parts or assemblies); the user could be invoking your add-on on several of these opened workspaces but there is only one loaded instance of the add-on DLL (and hence, only one `IAlibreAddOn` interface). So, how does your add-on DLL know which session is being referred to any point? This is where the “sessionId” comes in. Here it may be useful to just point out that using a method exposed in the AlibreX type-library API, add-on can resolve the “sessionId” to an `IADSession` interface pointer that wraps the workspace in question.

Step 4: Add a new class to implement `IAlibreAddOnCommand` interface to serve as a handler for every 'leaf' menu command item exposed by the AddOn. Let us call the new class `CAddOnCommand`. The definition of this class in the header file will look like this:

```
class CMyAddOnCommand : public IAlibreAddOnCommand
```

Implement required methods for `IUnknown` and `Idispatch`. This is usually boiler plate code (see sample addon project on how to implement this).

We will take a look at the main properties and methods in `IAlibreAddOnCommand` interface that a tightly integrated add-in may choose to implement:

```
// Methods from IAlibreAddOnCommand
HRESULT _stdcall putref_CommandSite(/* [in] */ IADAddOnCommandSite *pSite);

HRESULT _stdcall get_CommandSite(/* [retval][out] */ IADAddOnCommandSite **pSite);

HRESULT _stdcall AddTab(/* [retval][out] */ VARIANT_BOOL *pAddTab);

HRESULT _stdcall OnShowUI(/* [in] */ __int64 hWnd);

HRESULT _stdcall OnRender(/* [in] */ long hDC,
/* [in] */ long clipRectX, /* [in] */ long clipRectY,
/* [in] */ long clipWidth, /* [in] */ long clipHeight);

HRESULT _stdcall On3DRender (void);

HRESULT _stdcall OnClick (/* [in] */ long screenX, /* [in] */ long screenY,
/* [in] */ enum ADDONMouseButton buttons,
/* [retval] [out] */ VARIANT_BOOL *pIsHandled);

HRESULT _stdcall OnDoubleClick (/* [in] */ long screenX, /* [in] */ long screenY,
/* [retval] [out] */ VARIANT_BOOL *pIsHandled);

HRESULT _stdcall OnMouseDown (/* [in] */ long screenX, /* [in] */ long screenY,
/* [in] */ enum ADDONMouseButton buttons,
/* [retval] [out] */ VARIANT_BOOL *pIsHandled);

HRESULT _stdcall OnMouseMove (/* [in] */ long screenX, /* [in] */ long screenY,
/* [in] */ enum ADDONMouseButton buttons,
/* [retval] [out] */ VARIANT_BOOL *pIsHandled);

HRESULT _stdcall OnMouseUp (/* [in] */ long screenX, /* [in] */ long screenY,
/* [in] */ enum ADDONMouseButton buttons,
/* [retval] [out] */ VARIANT_BOOL *pIsHandled);

HRESULT _stdcall OnSelectionChange (void);

HRESULT _stdcall OnTerminate (void);
```

```

HRESULT _stdcall OnComplete( void);

HRESULT _stdcall OnKeyDown (/* [in] */ long keyCode,
/* [retval] [out] */ VARIANT_BOOL *pIsHandled);

HRESULT _stdcall OnKeyUp (/* [in] */ long keyCode,
/* [retval] [out] */ VARIANT_BOOL *pIsHandled);

HRESULT _stdcall IsTwoWayToggle( /* [retval][out] */ VARIANT_BOOL *pIsTwoWayToggle);

HRESULT _stdcall OnEscape ( /* [retval][out] */ VARIANT_BOOL *pIsHandled);

HRESULT _stdcall OnMouseWheel (/* [in] */ double delta,
/* [retval][out] */ VARIANT_BOOL *pIsHandled);

HRESULT _stdcall get_TabName(/*[out], [retval]*/ BSTR* pTabName);

HRESULT _stdcall get_Extents (/*[out,retval]*/ SAFEARRAY **pExtents);

```

When user clicks (invokes) an add-on's command, as noted earlier, Alibre Design will obtain the IAddOnCommand interface pointer from the add-on. It will then spawn off the following series of actions:

- Alibre Design will first call **IAlibreAddOnCommand.AddTab (*pAddTab)**.
If your command requires showing some GUI inside an add-on specific tab on the Design Explorer, your command needs to return "VARIANT_TRUE" in the function implementation of **AddTab ()** as shown below:

```

HRESULT _stdcall CMyAddOnCommand::AddTab (VARIANT_BOOL *pAddTab)
{
    // Return TRUE if we want this command to add a tab to Alibre's Explorer pane
    *pAddTab = VARIANT_TRUE;
    return S_OK;
}

```

- Upon receiving TRUE as shown above, AlibreDesign creates a new tab page on Design Explorer for this AddOn. Subsequently, it calls: **IAlibreAddOnCommand.OnShowUI (int hWnd)** which AddOn can implement to add any UI controls under its tab page using the passed handle for parenting purposes. For e.g., say AddOn wants to add a list control. Here is sample code for that:

```

HRESULT _stdcall CMyAddOnCommand::OnShowUI (__int64 hWnd)
{
    HRESULT hr = S_OK;
    CWnd* cWindow = CWnd::FromHandle ((HWND)hWnd);

    // Initialize List control
    CListCtrl* lstParNames = new CListCtrl ();
    lstParNames->Create(WC_CHILD | WS_VISIBLE | WS_BORDER | LVS_LIST,
        CRect(10, 10, 200, 200), cWindow, 1);

    // Add your list items here.
    ..
}

```

```

    ..
    return hr;
}

```

- Next, AlibreDesign checks whether this command is a two-way toggle command by calling **IAlibreAddOnCommand.IsTwoWayToggle(*plsTwoWayToggle)**. If add-on returns back TRUE value, **IAlibreAddOnCommand.OnComplete ()** is called next. AddOn can do any finishing work here. If it is not a two-way toggle command, AlibreDesign activates a 'Listener' to watch on user and canvas events/ updates. This listener will remain active until another tool/command is activated.
- Within Alibre Design, Alibre creates an object of type **IADAddOnCommandSite** for the activated command and passes it to add-on command via the property: **IAlibreAddOnCommand.putref_CommandSite (IADAddOnCommandSite *pSite)**. If the command wants to override rendering, this is one place to convey that information to AlibreDesign. For example, a suggested implementation is shown below

```

HRESULT _stdcall CMyAddOnCommand::putref_CommandSite(
    /* [in] */ IADAddOnCommandSite *pSite)
{
    AFX_MANAGE_STATE(AfxGetStaticModuleState());

    try
    {
        if (pSite)
        {
            // Store this command's command site for later use by our command
            pSite->QueryInterface (&m_pCmdSite);

            // Query Alibre to find out which rendering engine is currently
            // being used to render graphics -- the DirectX 9 based legacy
            // renderer or the new Hoops based renderer
            m_pCmdSite->LegacyRenderingEngine(&m_bLegacyRenderer);
        }
    }
    catch (...)
    {
        AfxMessageBox ("Exception in CMyAddOnCommand::putref_CommandSite");
    }
    return S_OK;
}

```

Note: The command site interface (**IADAddOnCommandSite**) exposes methods that the command could use during its lifetime to query useful information from Alibre Design.

- If AddOn command would like to respond to mouse moves or selection changes, necessary implementation could be provided for other event handling methods in IAlibreAddOnCommand interface.
- Alibre Design calls IAlibreAddOnCommand.Terminate () when the add-on command is terminated, say, as a result of user activating another command. It is worthy to note here that the add-on command could programmatically terminate itself by calling the **IADAddOnCommandSite.Terminate ()** method.